

Lab 3: Component Modelling and Architectural Pattern Selection

Second-Hand Product Escrow Marketplace

Samarth Mohan
SRN: PES1UG24CS703
Software Engineering

September 10, 2026

1 Scenario and Requirements

The scenario is the Second-Hand Product Escrow Marketplace from problem statement 39. A buyer pays for a used product, the money is held in escrow, an independent inspection agent certifies the condition of the product, and the money is released to the seller only after the buyer signs off. The requirements carried forward from Lab 1 are used as the input for the architecture:

- FR-001: lock the buyer payment in escrow and keep the order in the state Under Inspection until the inspector signs off.
- FR-002: let the buyer browse, order and pay through a payment gateway to fund the escrow.
- FR-003: let an inspection agent claim an order and upload a report with photos, a condition grade and remarks.
- FR-004: let the buyer read the report and either approve the release or raise a dispute with a reason.
- FR-005: release money to the seller only on approval and inspection pass, refund the buyer on failure or upheld dispute, and close the order.
- NFR-001: a dispute must freeze escrow funds at once, block withdrawals while the case is open, and leave an append only log.
- NFR-002: lock the escrow and update the inspection status in under 2 seconds for 500 concurrent users, with data encrypted in transit and at rest.

The main challenges follow from these requirements. Money correctness is the first, because a wrong release is a legal and trust problem. Latency is the second, because the lock and the dispute freeze both have a 2 second target. Trust of the inspection step is the third, because the buyer must be able to see evidence before money moves. The design below is judged against these three challenges.

2 Architectural Style Analysis

Style	Structure	Pros for this scenario	Cons for this scenario
Layered	Horizontal layers: presentation, business logic, data access, in one deployable unit.	Simple to build, simple to reason about, one deployment. Good fit when one team owns everything.	One deployable unit, so the busy browsing and photo upload path cannot be scaled without also scaling the money code. A slow report query in one layer can delay the escrow freeze, which breaks the NFR-001 target. Shared database couples the layers.
Microservices with an API Gateway	Small services that own their data and deploy separately, behind one gateway.	Independent scaling of the read heavy paths, fault isolation on the money path, payment code isolated with its own data, independent replacement of one service.	More deployment and monitoring work, network hops add small delays, events between services are eventually consistent and need a trace id to debug.
Client-Server	One central server that holds all logic and data, with thin clients.	Central control, one place for consistency rules, simple client.	The server is a single point of failure for fund operations, so a crash stops every lock, freeze and release at once, which directly contradicts NFR-001. Scaling means buying a bigger server, and there is no fault isolation.

Table 1: The three candidate styles judged against the escrow scenario.

2.1 Layered

A layered design is the cheapest to build and the easiest to explain. For this scenario its weak point is scaling. The listing pages, order status checks and inspection photo uploads carry most of the traffic, while the money operations are few. In one deployable unit we cannot add capacity to the busy path on its own. The same deployable also carries the payment and escrow code, so every release of a small listing change touches the code that moves money. A failure in a reporting query can also hold the threads that the escrow freeze needs, and the freeze has a 2 second target.

2.2 Microservices with an API Gateway

A microservice design splits the system along the rules it enforces: orders, payments, fund records, inspection work, notifications and identity. Each service owns its own data, so a change in the inspection service cannot damage the ledger. The cost is operational. There are more deployments, more logs and more network hops, and an event that fails between two services needs retries. This cost is worth paying here, because the money path is small and the rules around it are strict.

2.3 Client-Server

A client-server design gives one central place for the escrow rules, which is attractive on paper. It fails on availability. The marketplace holds buyer money, so the server must be up for every

lock, freeze, release and refund. A single server makes those operations stop together during any outage, a maintenance window or a load spike. NFR-001 asks for an immediate freeze during a dispute, and this design cannot promise that with one server, however well the server is written.

3 Architecture Selection and Justification

We chose Microservices with an API Gateway for the Second-Hand Product Escrow Marketplace.

3.1 Reasons

Reason one, independent scaling. Traffic in this scenario is uneven. Browsing and inspection photo uploads grow with the number of users, while escrow operations are few. NFR-002 sets a target of 500 concurrent users with the lock and status update under 2 seconds. With separate services we add copies of the gateway, the Order Manager and the Inspection Service when the crowd grows, and leave the Payment Service and Escrow Ledger at a small fixed size. In a single deployable unit the same growth would force copies of the whole application, including the money code.

Reason two, fault isolation on the money path. NFR-001 needs a dispute to freeze escrow funds at once and block withdrawals while the case is open. That freeze must not queue behind an email backlog or a slow photo upload. Because Notification Service and Inspection Service are separate services, neither can hold the freeze. The Order Manager reaches the Escrow Ledger directly, so the freeze still works when notifications are down.

3.2 Security Advantage

The Payment Service and the Escrow Ledger own their data, and no other service can write to the ledger. Card data never leaves the Payment Service, which is the only component that holds the payment gateway credential, so a weak spot in the inspection or notification code cannot expose card data or move money. The Web API Gateway is the only component that faces the internet, and calls between services are mutually authenticated. The ledger is append only, so every hold, freeze, release and refund is traceable and cannot be edited quietly. This supports the dual verification rule of FR-001.

3.3 Performance Benefit

Notifications, reminders and the 48 hour auto rule travel as asynchronous events, outside the purchase path. The synchronous chain for one purchase is short: gateway, Order Manager, Payment Service, Escrow Ledger. Only these four services sit inside the 2 second target of NFR-002, and each one can be scaled or tuned on its own.

3.4 Accepted Tradeoff

Microservices cost more to operate. There are more deployments to watch, more network hops, and event delivery between services is eventually consistent. We accept this because the escrow rules in this scenario are strict, and we reduce the cost by keeping the money path short, routing every client call through one gateway, and tagging every request with one trace id.

4 Components

Component	Responsibility	Requirement served
Buyer Web / Mobile App	Client used by the buyer to browse, order, fund escrow, read the inspection report, approve or dispute.	FR-002, FR-004
Inspection Agent Console	Client used by the inspection agent to list assigned orders and upload the report with photos and a grade.	FR-003
Web API Gateway	Single entry point. Ends TLS, checks the user token, limits request rate and routes to the right service.	NFR-002, security
Order Manager	Owns the order state machine and coordinates lock, inspection, approve, dispute, release and refund.	FR-001, FR-004, FR-005
Payment Service	Holds the payment gateway credential, creates idempotent fund requests, locks, releases and refunds money.	FR-002, FR-005
Escrow Ledger	Append only record of every hold, freeze, release and refund. Single authority for the funds state.	FR-001, NFR-001
Inspection Service	Creates inspection tasks, stores the report with photos, grade and remarks, records the sign-off.	FR-003
Photo Object Store	Holds the inspection photos. The Inspection Service issues presigned upload and read requests against it.	FR-003
Notification Service	Sends buyer and agent notices, 48 hour reminders and the auto rule result as asynchronous events.	FR-004 time-out path
Identity and Access	Login, token issue and introspection, verification status of buyer and seller.	Security for all FRs
External Payment Gateway	Third party card processor that authorizes and captures the payment. Outside our control.	FR-002

Table 2: Components of the marketplace.

5 Interfaces

Interface	Provider	Consumer	Protocol	Purpose
IClientAPI	Web API Gateway	Buyer app, Agent console	HTTPS/REST with JWT	One entry point for all client calls
IOrderAPI	Order Manager	Web API Gateway	REST/JSON over mTLS	Place order, fund escrow, approve, dispute
IAgentAPI	Inspection Service	Web API Gateway	REST/JSON over mTLS	Agent lists tasks and submits the report
IAuth	Identity and Access	Web API Gateway	OAuth2 token introspection	Verify token and role before routing
IPayment	Payment Service	Order Manager	REST/JSON with idempotency key	Lock, release and refund funds
IEscrowLedger	Escrow Ledger	Order Manager, Payment Service	gRPC, append only writes	Record hold, freeze, release and refund
IInspectionWork	Inspection Service	Order Manager	REST/JSON with presigned upload	Create task, return sign-off
INotify	Notification Service	Order Manager	AMQP asynchronous events	Status notices, reminders, auto rule
IPSPGateway	External Payment Gateway	Payment Service	HTTPS, tokenized	Card authorization and capture

Table 3: Interfaces with provider, consumer, protocol and purpose.

One dependency in the diagram is not an interface. The Inspection Service uses the Photo Object Store through the store’s own HTTPS API for presigned photo upload and reads, so no interface is modelled between them. It is drawn as the one dashed arrow in the diagram, carrying the «use» stereotype, with the arrow head pointing at the Photo Object Store.

6 Component Diagram

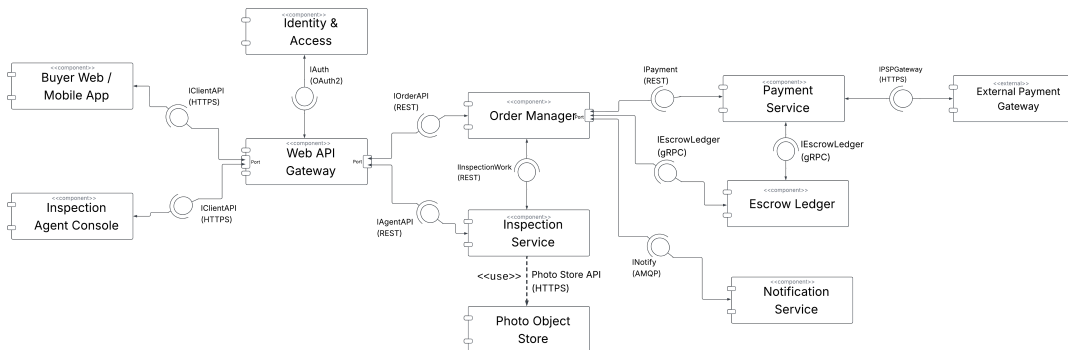


Figure 1: Component diagram of the Second-Hand Product Escrow Marketplace under the Microservices with API Gateway architecture. Every solid connector is an assembly connector, with the ball on the component that provides the interface and the socket on the component that requires it. Ports group the interfaces that meet at one interaction point, and the single dashed arrow is the one usage dependency.

7 Conclusion

The escrow scenario needs three things at the same time: correct money movement, a fast freeze and lock path, and a way to scale the read heavy parts without touching the money code. Microservices with an API Gateway gives all three, at the cost of more deployment work. The component diagram and the tables above show how the services are split, which interfaces they provide, which interfaces they require, the protocol used on each interface, and the one dependency that is not an interface.